V3 GRAND MADURAI

Investment Lifecycle Platform

System Architecture & Platform Blueprint

Event-Driven Architecture | Service-Oriented Engines | Full Lifecycle Tracking

Modeled on ARGUS Developer + Aprao + Built/Northspyre + Procore

March 2026 | Version 2.0

CONFIDENTIAL

Table of Contents

1. Platform Vision

This document describes the architecture of a living investment operating system — not a static calculator, not a one-time underwriting tool, and not an HTML dashboard. The system is designed as a long-lived, event-driven, lifecycle platform that continuously tracks, recomputes, and manages real estate investments from initial idea through grand opening and early operations.

1.1 What This System Is

A persistent, event-driven platform where deals, assumptions, RFIs, change orders, risks, and progress updates are stored in a durable data store and versioned. Engines re-run automatically when events occur (new macro data, design change, permit delay, budget variance), and recommendations update over time.

An end-to-end lifecycle tracker that adapts across pre-development, underwriting, design, financing, construction (with Procore-style tracking), pre-opening, and post-opening performance. Each phase has defined data entities, workflows, triggers, alerts, and an evolving recommendation/risk view.

A service-oriented engine platform where Underwriter, Monte Carlo, Factor Engine, Decision Engine, Budget Variance Engine, and S-Curve Engine are standalone services with clear APIs. The React UI is just one client — the logic lives in services, not in the browser.

An adaptive recommendation system that maintains a live recommendation state per deal — INVEST / HOLD / DE-RISK / EXIT / DO-NOT-PROCEED — with confidence scores that evolve as lifecycle events arrive.

1.2 What This System Is Not

It is not a static HTML page with embedded JavaScript engines. It is not a spreadsheet replacement. It is not a one-time underwriting tool that produces a PDF and forgets. It is not coupled to hotels — the first implementation is a hotel in Madurai, but the platform is asset-agnostic by design.

1.3 Design Influences

Platform	Influence Taken
ARGUS Developer	Multi-stage development pro-forma, S-curve cash flow distribution, multi-tranche financing, institutional-grade sensitivity analysis
Aprao	Fast scenario-driven feasibility, residual land value, instant re-computation on any input change, investor-ready report generation
Lead Developer / Feasibility.pro	Multi-stage development feasibility, S-curve capex phasing, multiple capital tranches across the full project timeline
Built / Northspyre	Connecting underwriting through construction to operations in one system; budget vs actuals, draw requests, variance analysis, governance and audit
Procore	RFIs, change orders, cost code structure, budget line items, schedule tracking, milestone management, the RFI-to-change-order workflow chain
Hotel-Specialist Practices	USALI departmental P&L, RevPAR/ADR/occupancy ramp modeling, management/franchise fees, FF&E reserves, segment mix, seasonal demand patterns

2. Domain Model

The domain model defines every persistent entity in the system and how they relate. This is the schema that the durable data store enforces. All engines, all workflows, and all UI views read from and write to this model.

2.1 Entity Relationship Overview

The system is organized around a central Deal aggregate root. Every other entity is owned by or references a Deal.

```
Deal ──┬── Property ─── RoomTypes[], Amenities[]
      ├── Partnership ── Partners[]
      ├── MarketAssumptions ── Segments[], Seasonality[], CompSet[]
      ├── FinancialAssumptions
      ├── CapexPlan ─── CapexPhases[] ─── CapexLineItems[]
      ├── OpexModel ─── DepartmentExpenses[] (USALI)
      ├── AnchorContracts[] ─── MOUs[]
      ├── Scenarios[] (Bear/Base/Bull + custom)
      ├── LifecyclePlan ─── Phases[] ─── Tasks[] ─── Milestones[]
      ├── ConstructionTracker ──┬── BudgetLines[]
      │                         ├── RFIs[] ─── ChangeOrders[]
      │                         ├── DrawRequests[]
      │                         └── ScheduleTracker ─── Milestones[]
      ├── FinancingPlan ─── DebtTranches[] ─── DrawdownSchedule[]
      ├── RiskRegister ─── Risks[] ─── Mitigations[]
      ├── RecommendationHistory[] (versioned INVEST/HOLD/DE-RISK/EXIT)
      ├── EngineResults[] (versioned outputs from each engine run)
      ├── Reports[] (generated investment memos, lender packages, etc.)
      └── AuditLog[] (immutable, append-only)
```

2.2 Core Entities

2.2.1 Deal

The aggregate root. Every operation in the system is scoped to a Deal. A Deal has a lifecycle status that progresses through defined phases.

Field	Type	Description
id	UUID	Unique deal identifier
name	String	Human-readable deal name (e.g., "V3 Grand Madurai")
assetClass	Enum	hotel | multifamily | office | retail | industrial | mixed-use
subClass	String	Asset-specific subtype (e.g., "4-star-boutique")
status	Enum	pipeline | underwriting | approved | construction | pre-opening | operating | exited
lifecyclePhase	Enum	Current active phase (see Lifecycle section)
currentMonth	Int	Months since project inception (T=0)

Field	Type	Description
createdAt	Timestamp	Deal creation date
updatedAt	Timestamp	Last modification (any child entity)
version	Int	Monotonically increasing version counter

2.2.2 Property

Field	Type	Description
location	String + GeoPoint	Address and lat/lng
landArea	Decimal + Unit	Land parcel size
grossBUA	Decimal + Unit	Gross built-up area
keys / units	Object	Per-phase count (e.g., phase1: 55, phase2: 17)
roomTypes[]	Array	Type, count, sqft, phase assignment, rate tier
amenities[]	Array	Facility list with capex and opex implications

2.2.3 CapexPlan and CapexLineItem

Models the ARGUS Developer-style phased capital expenditure with S-curve distribution.

Field	Type	Description
phase	Enum	phase1 | phase2 | phaseN
totalBudget	Decimal	Total capex for this phase (e.g., 35.7 Cr)
items[]	Array<CapexLineItem>	Individual cost code line items
item.code	String	Cost code (LAND, CIVIL, MEP, INTD, FFE, PREOP, CONT)
item.amount	Decimal	Budgeted amount for this line item
item.startMonth	Int	Month when spend begins (relative to T=0)
item.endMonth	Int	Month when spend ends
item.curveType	Enum	front-loaded | s-curve | back-loaded | linear
item.committed	Decimal	Committed (contracted) amount — updated live
item.spent	Decimal	Actual spend to date — updated live
item.forecast	Decimal	Forecast at completion — re-computed on events

2.2.4 ConstructionTracker

Procore-inspired construction management entities.

BudgetLine: Each cost code has a budget, committed, spent, and forecast amount. Variance is continuously tracked.

RFI (Request for Information): Raised when a design ambiguity or site condition requires clarification. Has cost impact estimate, schedule impact (days), assignee, status (open/in-review/closed), and can spawn ChangeOrders.

ChangeOrder: Created from an RFI or independently. Has an amount, approval status (pending/approved/rejected), reference to originating RFI, and updates the budget line forecast when approved.

DrawRequest: Periodic request for construction financing disbursement. Includes current period amount, retainage, cumulative draws, lender approval status, and required documentation checklist.

ScheduleTracker: Planned vs actual dates for each milestone. Computes days variance, percent complete, and earned value metrics.

2.2.5 FinancingPlan and DebtTranches

Field	Type	Description
debtRatio	Decimal	Overall debt proportion (e.g., 0.40)
tranches[]	Array	Each tranche: lender, amount, rate, tenure, moratorium, drawdown schedule, covenants
equitySchedule	Object	Equity injection timeline per partner
covenants[]	Array	DSCR minimums, LTV maximums, reporting requirements
drawdownSchedule[]	Array	Monthly planned vs actual draws per tranche

2.2.6 RecommendationState

The live recommendation is a versioned, append-only record that evolves over the deal lifecycle.

Field	Type	Description
version	Int	Auto-incremented on each re-evaluation
timestamp	Timestamp	When this recommendation was computed
recommendation	Enum	INVEST | HOLD | DE-RISK | EXIT | DO-NOT-PROCEED
confidence	Int (0-100)	Composite confidence score
triggerEvent	String	The event that caused this re-evaluation
gateResults[]	Array	Individual gate criterion results (pass/fail/detail)
factorSnapshot	Object	Factor scores at time of evaluation
proformaSnapshot	Object	Key pro-forma metrics (IRR, NPV, DSCR, MOIC)
mcSnapshot	Object	Monte Carlo distribution summary (P10/P50/P90)
explanation	String	Natural-language rationale for the recommendation
flipConditions[]	Array	What would change the recommendation up or down

2.2.7 AuditLog

Immutable, append-only. Every mutation to any entity in the system produces an audit entry.

Field	Type	Description
id	UUID	Unique log entry ID
timestamp	Timestamp	When the event occurred
dealId	UUID	Which deal was affected

Field	Type	Description
userId	UUID	Who performed the action
role	Enum	Role of the acting user
module	String	Which system module (underwriting, construction, risk, etc.)
action	String	What happened (human-readable)
entityType	String	Entity class affected (CapexLineItem, RFI, Risk, etc.)
entityId	UUID	ID of the specific entity
diff	JSON	Before/after snapshot of changed fields

3. Event & Workflow Model

The system is event-driven. Every meaningful change produces an event. Events are published to a message bus. Engine services subscribe to events they care about and re-compute their outputs. The orchestration layer sequences multi-step workflows.

3.1 Event Taxonomy

Events are categorized by the lifecycle domain they originate from. Every event carries: eventId, dealId, timestamp, source (user | system | external), and a typed payload.

Domain	Event	Payload	Triggered By
Assumptions	assumption.updated	path, oldValue, newValue	User edits any assumption
Assumptions	macro.refreshed	all macro indicators	Scheduled daily pull from market data feeds
Lifecycle	phase.advanced	phaseId, newStatus	PM marks phase transition
Lifecycle	milestone.completed	milestoneId, actualDate	PM records milestone achievement
Lifecycle	milestone.delayed	milestoneId, newDate, daysSlipped	Schedule update shows slippage
Construction	budget.actual.updated	costCode, newSpent, newForecast	Invoice processed or actuals uploaded
Construction	rfi.created	RFI object	PM or consultant raises RFI
Construction	rfi.resolved	rfiId, resolution, costImpact	Assignee closes RFI
Construction	change-order.submitted	CO object	PM submits CO from RFI resolution
Construction	change-order.approved	coId, amount	Authorized approver signs off
Construction	draw-request.submitted	DrawRequest object	Monthly draw request filed
Construction	draw-request.approved	drawId, amount	Lender approves disbursement
Financing	covenant.breached	covenantId, actual, threshold	Engine detects covenant violation
Financing	tranche.drawn	trancheId, amount	Funds disbursed from lender
Risk	risk.escalated	riskId, newSeverity	Risk probability or impact increased
Risk	risk.mitigated	riskId, mitigation	Risk action completed
Operations	actuals.reported	month, revenue, occupancy, adr, ebitda	Monthly operating data uploaded
Operations	phase2-gate.evaluated	criteriaResults[], decision	Month 36 assessment triggered

Domain	Event	Payload	Triggered By
Engine	recommendation.changed	oldRec, newRec, triggerEvent	Decision Engine produces new recommendation
Engine	engine.completed	engineName, version, summary	Any engine finishes a computation cycle

3.2 Event Flow Architecture

```
[User Action / External Feed / Timer]
            |
            v
    +----------------+
    |   Event Bus    |  (Kafka / NATS / Redis Streams)
    +-------+--------+
            |
   +--------+--------+--------+--------+--------+
   v        v        v        v        v        v
 Factor   Under-   Monte    Budget   S-Curve  Decision
 Engine   writer   Carlo    Variance Engine   Engine
   |        |        |        |        |        |
   v        v        v        v        v        v
    +----------------+------------------+
    |         Result Store              |
    |   (versioned engine outputs)      |
    +------------------+----------------+
                       |
                       v
              Recommendation State
              (INVEST/HOLD/DE-RISK/EXIT)
                       |
              +--------+--------+
              v        v        v
           Alerts   Dashboard  Reports
```

3.3 Engine Trigger Matrix

Not every event triggers every engine. The trigger matrix defines which events cause which engines to re-run.

Event	Factor	Underwriter	Monte Carlo	Budget Var.	S-Curve	Decision
assumption.updated	If macro	Yes	Yes	—	If capex timing	Yes
macro.refreshed	Yes	Yes	Yes	—	—	Yes

Event	Factor	Underwriter	Monte Carlo	Budget Var.	S-Curve	Decision
budget.actual.updated	—	—	—	Yes	—	If variance > 5%
change-order.approved	—	Yes (rebase)	Yes	Yes	Yes (re-phase)	Yes
milestone.delayed	—	If > 30 days	—	—	Yes (re-phase)	If > 30 days
actuals.reported	—	Yes (blend)	Yes	—	—	Yes
covenant.breached	Yes (+risk)	Yes	Yes	—	—	Yes (force)
risk.escalated	Yes	—	Yes	—	—	Yes
phase2-gate.evaluated	—	Yes	Yes	—	Yes	Yes

3.4 Workflow Orchestration

Multi-step workflows are managed by a Temporal-style orchestration engine. Workflows are durable, resumable, and support compensation (rollback) on failure.

3.4.1 Example: Change Order Approval Workflow

```
Workflow: ChangeOrderApproval
  Trigger: change-order.submitted
  Steps:
    1. Validate CO data (amount, RFI ref, cost code)
    2. Route to approver based on amount threshold:
       - < 0.10 Cr: Project Manager
       - 0.10-0.50 Cr: Lead Investor
       - > 0.50 Cr: All Partners (majority vote)
    3. Wait for approval (with 72-hour escalation timer)
    4. On approval:
       a. Update BudgetLine forecast
       b. Emit change-order.approved event
       c. Trigger engine cascade (Underwriter -> MC -> Decision)
       d. If recommendation changes: emit alert
    5. On rejection:
       a. Notify submitter with rejection reason
       b. Emit change-order.rejected event
```

3.4.2 Example: Monthly Revaluation Workflow

```
Workflow: MonthlyRevaluation
  Trigger: Timer (1st of each month) OR actuals.reported
  Steps:
    1. Pull latest macro indicators (RBI repo, CPI, bond yields)
    2. Emit macro.refreshed event
    3. Wait for Factor Engine completion
```

```
4. Run Underwriter with latest actuals blended into projections
5. Run Monte Carlo (3000 iterations)
6. Run Decision Engine with all fresh inputs
7. Compare new recommendation vs previous:
   - If changed: emit recommendation.changed + alert to all roles
   - If confidence delta > 10 pts: emit alert to lead investor
8. Store versioned engine results and recommendation snapshot
```

4. Service Architecture

Every computation engine is a standalone service with a defined API contract. Services are stateless — they read from the data store, compute, and write results back. The event bus decouples producers from consumers.

4.1 High-Level Architecture

```
+=====================================================================+
|                        CLIENT LAYER                                 |
|  React Web App | Mobile App | Investor Portal | Lender Portal      |
+=====================================================================+
                               |
                     API Gateway (REST + WebSocket)
                               |
+=====================================================================+
|                     ORCHESTRATION LAYER                             |
|  Temporal / Step Functions: Durable Workflows + Saga Patterns       |
+=====================================================================+
       |          |          |          |          |          |
+------+--+ +-----+---+ +----+----+ +--+-------+ ++-------+ ++------+
| Factor   | | Under-  | | Monte   | | Budget   | | S-Curve| | Decis-|
| Engine   | | writer  | | Carlo   | | Variance | | Engine | | ion   |
| Service  | | Service | | Service | | Service  | | Service| | Engine|
+----+-----+ +----+----+ +----+----+ +----+-----+ +---+----+ +---+--+
     |            |            |            |           |          |
+=====================================================================+
|                        EVENT BUS                                    |
|              (Kafka / NATS / Redis Streams)                         |
+=====================================================================+
                               |
+=====================================================================+
|                        DATA LAYER                                   |
|  PostgreSQL        | Redis Cache    | S3 / Blob                     |
|  (domain model,    | (live engine   | (generated reports,           |
|   audit log,       |  results,      |  document store,              |
|   event store)     |  sessions)     |  attachments)                 |
+=====================================================================+
```

4.2 Engine Service APIs

4.2.1 Factor Engine Service

Endpoint	Method	Input	Output
/factor/score	POST	dealId + macro overrides (optional)	{ domains: {global, local, asset, sponsor}, composite, requiredReturn, factors: {...} }

Endpoint	Method	Input	Output
/factor/sensitivity	POST	dealId + parameterToVary + range	{ sensitivityTable: [{paramValue, composite, requiredReturn}] }
/factor/history	GET	dealId	{ versions: [{timestamp, composite, requiredReturn, trigger}] }

Subscribes to: macro.refreshed, assumption.updated (if macro path), risk.escalated, covenant.breached

4.2.2 Underwriter Service

Endpoint	Method	Input	Output
/underwrite/proforma	POST	dealId + scenarioKey (or custom overrides)	10-year pro-forma: {years[], cashFlows[], irr, npv, moic, payback, dscr[], exitValue}
/underwrite/scenarios	POST	dealId	All scenarios: {bear, base, bull, custom[], weighted}
/underwrite/sensitivity	POST	dealId + 2 parameters + ranges	Sensitivity matrix: {grid[][]: irr values for each combination}
/underwrite/rebase	POST	dealId + actuals[] + changeOrders[]	Rebased pro-forma incorporating actual performance and approved COs

Subscribes to: assumption.updated, change-order.approved, actuals.reported, milestone.delayed (if > 30 days)

4.2.3 Monte Carlo Service

Endpoint	Method	Input	Output
/mc/simulate	POST	dealId + iterations (default 3000) + distributions config	{irr: {p10-p90, mean}, npv: {p10-p90}, probNPVNeg, probBelowHurdle, histogram, convergenceMetric}
/mc/stress	POST	dealId + stressScenario (recession, rate-shock, etc.)	{stressedDistribution, conditional probabilities}

Subscribes to: assumption.updated, macro.refreshed, change-order.approved, actuals.reported, risk.escalated

4.2.4 Budget Variance Service

Endpoint	Method	Input	Output
/budget/analyze	POST	dealId	{lines[]: {code, budget, committed, spent, forecast, variance, status}, totals, coImpact, alerts[]}
/budget/forecast	POST	dealId + earnedValueData	{forecastCompletion, estimateAtCompletion, varianceAtCompletion, CPI, SPI}
/budget/draw-request	POST	dealId + period + lineItems[]	{drawAmount, retainage, cumulativeDrawn, documentation[]}

Subscribes to: budget.actual.updated, change-order.approved, draw-request.approved

4.2.5 S-Curve Engine Service

Endpoint	Method	Input	Output
/scurve/distribute	POST	capexItems[] + curveParams	{monthlyCashFlow[], cumulativeCurve[], peakSpendMonth}
/scurve/rebase	POST	dealId + approvedCOs + scheduleDelays	{adjustedCurve[], newPeakMonth, deltaFromOriginal[]}

Subscribes to: change-order.approved, milestone.delayed, assumption.updated (if capex timing)

4.2.6 Decision Engine Service

Endpoint	Method	Input	Output
/decision/evaluate	POST	dealId (reads latest engine results from store)	{recommendation, confidence, gateResults[], explanation, flipConditions[], alertLevel}
/decision/explain	GET	dealId + version (optional)	{narrative: human-readable explanation of current recommendation and what would change it}
/decision/history	GET	dealId	{versions[]: [{timestamp, recommendation, confidence, trigger}]}

Subscribes to: engine.completed (from any other engine), covenant.breached, phase2-gate.evaluated. The Decision Engine is always the last to run in any cascade.

4.3 Data Stores

Store	Technology	Purpose	Retention
Deal Store	PostgreSQL	All domain entities (deals, properties, budgets, RFIs, risks)	Permanent
Event Store	PostgreSQL + Kafka	All events with full payload, ordered by timestamp	Permanent (event sourcing)
Engine Result Store	PostgreSQL + Redis	Versioned outputs from each engine run; Redis for latest	Permanent (PG) + Latest (Redis)
Audit Log	PostgreSQL (append-only)	Every mutation with before/after diff	Permanent, immutable
Document Store	S3 / Azure Blob	Generated reports (PDFs, DOCX), RFI attachments, drawings	Permanent
Session Cache	Redis	User sessions, active computation state, real-time subscriptions	24-hour TTL

5. Lifecycle Phases & Adaptive Recommendation

The system tracks 9 lifecycle phases. At each phase, the recommendation state can evolve. This section defines what data arrives at each phase, what engines run, and how the recommendation adapts.

5.1 Phase Definitions

Phase	Duration	Key Data Entities	Primary Engines	Recommendation States
1. Feasibility	Months -3 to 0	Market study, site evaluation, partnership terms, preliminary financial model	Factor, Underwriter, MC	INVEST or DO-NOT-PROCEED
2. Land Acquisition	Months 0-1	Purchase agreement, title search, land cost, due diligence findings	Underwriter (land basis), Factor	INVEST or EXIT (if due diligence fails)
3. Entitlements	Months 1-6	Permit applications, zoning approvals, environmental clearance, fire NOC	Factor (regulatory risk), Decision	HOLD if permits delayed, INVEST if on track
4. Design	Months 3-8	Architectural drawings, MEP design, interior specifications, cost estimates	S-Curve, Underwriter (rebase with design costs)	INVEST or DE-RISK if costs exceed budget
5. Financing	Months 4-6	Term sheets, loan sanction, equity schedule, insurance, covenants	Underwriter (with actual terms), MC, Decision	INVEST or HOLD if financing terms unfavorable
6. Construction	Months 7-26	Budget actuals, RFIs, change orders, draw requests, schedule milestones	Budget Variance, S-Curve, Underwriter (rebase), MC, Decision	INVEST | HOLD | DE-RISK depending on variance
7. FF&E Procurement	Months 18-27	PO commitments, delivery tracking, installation progress	Budget Variance	Inherits from Construction
8. Pre-Opening	Months 24-28	Staff hiring progress, training completion, systems setup, soft opening metrics	Factor (operational readiness), Decision	INVEST or DE-RISK if readiness gaps
9. Operations	Months 28+	Monthly P&L, occupancy, ADR, RevPAR, guest ratings, MOU realization	Underwriter (actuals blend), MC, Factor, Decision	INVEST | HOLD | DE-RISK | EXIT | INVEST-MORE

5.2 Adaptive Recommendation Logic

The recommendation is not a one-time decision. It is a continuously evolving state that responds to lifecycle events. The five possible states are:

State	Meaning	Triggered When
INVEST	All gates pass, confidence > 65, no active high-severity risks	Initial approval and sustained through lifecycle

State	Meaning	Triggered When
HOLD	Some uncertainty exists; wait for more data before proceeding	Key metrics within 150bps of thresholds; permits delayed; financing terms pending
DE-RISK	Active intervention required to protect returns	Cost overrun > 5% of budget; DSCR approaching covenant; schedule > 30 days late; P(NPV<0) rising
EXIT	Fundamental thesis broken; recommend unwinding position	IRR drops below risk-free rate; new competitor materializes; regulatory block; partnership dispute
DO-NOT-PROCEED	Initial screening rejects the deal	Composite factor score < 2.25; P(NPV<0) > 30%; bear IRR < risk-free rate

5.2.1 State Transition Rules

Transitions are triggered by events and validated by the Decision Engine. Key transitions:

Construction cost overrun > 5%: INVEST → DE-RISK. The Decision Engine recommends specific corrective actions: value engineering, scope reduction, contingency draw, or equity call.

Schedule delay > 30 days: INVEST → HOLD. Underwriter re-phases the pro-forma. If the delay pushes IRR below required return: HOLD → DE-RISK.

Positive operating actuals (post-opening): If actual occupancy/ADR exceed pro-forma for 3 consecutive months: confidence increases. If Phase 2 gate criteria met: INVEST → INVEST-MORE (recommend Phase 2 expansion).

Covenant breach: Immediate DE-RISK with specific remediation plan. If not cured within 90 days: DE-RISK → EXIT.

Macro environment shift: If RBI hikes repo rate > 150bps or GDP growth drops below 5%: Factor Engine re-scores, which may cascade to HOLD or DE-RISK depending on magnitude.

5.3 Alert Rules by Role

Alert Condition	Lead Investor	Co-Investor	Operator/GM	Lender	PM
Recommendation changed	Immediate	Immediate	Immediate	If DE-RISK/EXIT	—
Confidence dropped > 10pts	Immediate	Daily digest	—	If < 50	—
Budget variance > 5%	Immediate	Weekly	—	Immediate	Immediate
Schedule delay > 15 days	Daily digest	Weekly	—	If > 30 days	Immediate
RFI open > 14 days	—	—	—	—	Immediate
Change order > 0.5 Cr	Immediate (approval)	Immediate	—	Immediate	Immediate
DSCR < 1.30x projected	—	—	—	Immediate	—
Monthly actuals available	Summary report	Summary report	Detail report	Summary + DSCR	—
Covenant breach risk	Immediate	Immediate	Immediate	Immediate	—

Alert Condition	Lead Investor	Co-Investor	Operator/GM	Lender	PM
Phase 2 gate assessment due	Immediate	Immediate	Immediate	—	Immediate

6. Role & Permissions Model

The system enforces role-based access control (RBAC) at the API layer. Every request carries a role context. Permissions are defined per module and per action type.

6.1 Role Definitions

Role	Description	Typical User
lead-investor	Full read/write access to all modules. Can approve change orders, modify assumptions, change scenarios. Owns the deal.	Partner 1 (50% equity)
co-investor	Read access to all modules. Can comment and flag issues. Cannot modify assumptions or approve COs above threshold.	Partner 2 (25% equity)
operator	Read/write on construction, operations, risk. Can raise RFIs, submit COs, report actuals. Cannot modify financial assumptions.	Partner 3 (GM/Operator, 25% equity)
lender	Read-only on underwriting, construction budget, risk, DSCR. Receives covenant alerts. Can approve draw requests.	SBI Term Loan Officer
project-manager	Read/write on construction, lifecycle tasks. Can raise RFIs, submit COs, update schedule. No financial model access.	Construction PM / Site Engineer

6.2 Permission Matrix

Module	Lead Investor	Co-Investor	Operator	Lender	PM
Assumptions	Read/Write	Read	Read	—	—
Underwriting	Read/Write	Read	Read	Read (limited)	—
Monte Carlo	Read/Run	Read	Read	—	—
Factor Scoring	Read/Write	Read	Read	—	—
Decision	Read	Read	Read	Read (limited)	—
Lifecycle	Read/Write	Read	Read/Write	Read	Read/Write
Construction Budget	Read/Approve	Read	Read/Write	Read	Read/Write
RFIs	Read/Approve	Read	Read/Write/Create	Read	Read/Write/Create
Change Orders	Read/Approve	Read	Read/Submit	Read	Read/Submit
Draw Requests	Read/Submit	Read	Read	Read/Approve	Read
Risk Register	Read/Write	Read	Read/Write	Read	Read/Write
Reports	Generate/Read	Read	Generate/Read	Read	Read
Audit Log	Read	Read	Read	Read (own)	Read (own)
Settings	Full	—	Limited	—	—

7. How UI Clients Consume Services

The UI layer is a thin client that consumes the engine services via REST APIs and subscribes to real-time updates via WebSocket. The UI contains zero computation logic — it is purely a rendering and interaction layer.

7.1 Client Architecture

```
+-----------------------------------------------------+
|              UI Client (React/Next.js)              |
|                                                      |
|  +----------+  +----------+  +-----------+         |
|  | Decision |  | Underwr. |  | Construct |  ...    |
|  | View     |  | View     |  | View      |         |
|  +----+-----+  +----+-----+  +-----+-----+        |
|       |             |               |               |
|  +----+-------------+---------------+-----+         |
|  |         State Management (Zustand)     |         |
|  |    server state via React Query/SWR    |         |
|  +----+---------------------------+-------+         |
|       |                           |                 |
|  +----+-------+           +-------+--------+        |
|  | REST Client|           | WebSocket Sub  |        |
|  | (queries)  |           | (real-time)    |        |
|  +----+-------+           +-------+--------+        |
+-------|-----------------------|-------------------+
        |                       |
   API Gateway              Event Stream
```

7.2 Data Flow Pattern

1. Initial Load: UI calls GET /deals/{dealId}/summary which returns the latest recommendation, key KPIs, and phase status. No computation happens client-side.

2. Module View: When user navigates to (e.g.) the Underwriting module, UI calls GET /deals/{dealId}/underwriting/latest which returns the most recent engine result from the Result Store. The UI renders the pre-computed pro-forma table, charts, and metrics.

3. User Edits Assumption: UI sends PATCH /deals/{dealId}/assumptions with the changed field. The API Gateway validates, persists, emits assumption.updated event, and returns 202 Accepted. The UI shows a "re-computing..." indicator.

4. Engine Cascade: The event triggers the engine cascade per the trigger matrix. Each engine completes and emits engine.completed. The Decision Engine runs last and emits recommendation.changed if the recommendation changes.

5. Real-Time Update: The UI receives the engine.completed and recommendation.changed events via WebSocket. It fetches the updated results and re-renders the affected views. The user sees the new IRR, recommendation, and confidence score update in real time.

6. Reports: User clicks "Generate Investment Memo". UI calls POST /deals/{dealId}/reports/investment-memo. The Report Service pulls the latest engine results, applies a template, generates a DOCX/PDF, stores it in the Document Store, and returns a download URL.

7.3 Offline / Degraded Mode

The UI caches the latest engine results locally. If the connection to the backend is lost, the UI continues to display the last-known state with a "stale data" indicator. When reconnection occurs, it fetches all events since last sync and updates incrementally.

8. Productization & Asset-Class Abstraction

The platform is designed with explicit separation between the generic lifecycle/engine layer and asset-specific logic. This section defines the abstraction boundaries.

8.1 Layer Separation

Layer	Generic (all asset classes)	Hotel-Specific (V3 Grand)
Domain Model	Deal, Property, CapexPlan, FinancingPlan, RiskRegister, AuditLog, RecommendationState	RoomTypes, Segments (medical/corporate/leisure), Seasonality, USALI OpexModel, AnchorContracts, Phase2Gate
Factor Engine	4-domain weighted scoring framework, configurable factors per domain, composite score calculation	Hotel factors: SubmarketRevPAR, AirportProximity, MedicalCorridor, CompSet, SeasonalDemand, BrandModel
Underwriter	10-year pro-forma structure, IRR/NPV/MOIC/DSCR, debt service, S-curve capex	USALI P&L, RevPAR/ADR/Occupancy ramp, departmental revenue (rooms/F&B/other), FF&E reserve, management fees
Monte Carlo	Distribution framework, Cholesky correlation, convergence metrics	Hotel-specific distributions: Occ~Triangular, ADR~LogNormal, OccADR correlation rho=-0.40
Decision Engine	6-criteria gate framework, confidence scoring, flip conditions, state machine	Hotel gate thresholds (e.g., DSCR 1.25x, P(NPV<0) 10%), Phase 2 gate criteria
Construction	Budget lines, RFIs, COs, draw requests, schedule — fully generic	Cost code taxonomy (CIVIL, MEP, INTD, FFE, PREOP)
Lifecycle	9-phase framework, task/milestone tracking — fully generic	Phase names, task lists, milestone definitions specific to hotel development

8.2 Adding a New Asset Class

To add a new asset class (e.g., multifamily residential, industrial warehouse, data center), a developer would:

1. Define asset-specific sub-entities (unit types, tenant profiles, lease structures) as extensions to the Property model.

2. Configure the Factor Engine with asset-specific factors (e.g., for multifamily: rent growth, vacancy rate, cap rate trend, rent-to-income ratio, walkability score).

3. Implement an asset-specific revenue model that plugs into the Underwriter (e.g., for multifamily: rent roll, lease expiry schedule, renewal probability, tenant improvement costs).

4. Configure Monte Carlo distributions appropriate to the asset class (e.g., for multifamily: Vacancy~Triangular, RentGrowth~Normal, CapRate~LogNormal).

5. Set Decision Engine gate thresholds appropriate to the asset class risk profile.

The lifecycle phases, construction tracking, event bus, audit trail, role model, and report generation layer require zero changes.

9. V3 Grand Madurai — First Implementation

The first concrete implementation of this platform is the V3 Grand Madurai project: a 72-key (55+17 phased) 4-star hotel near Madurai Airport, Tamil Nadu, India. Total investment: ₹43.0 Crore. Partnership: 50/25/25.

9.1 Current System State (Month 14)

Metric	Value	Status
Lifecycle Phase	Construction (Month 14 of 26)	Active
Recommendation	INVEST	Green
Confidence	69/100	Moderate — driven by Monte Carlo IQR width
Base IRR	16.7%	Above required return (13.3%)
NPV @ 12%	₹11.4 Crore	Positive
Factor Composite	4.20 / 5.00	Strong
P(NPV < 0)	8.7%	Below 10% threshold
Construction % Complete	34%	On track
Budget Variance	+₹0.50 Cr (under budget)	Green
Open RFIs	2	Watch — RFI-003 and RFI-004 pending
Approved Change Orders	₹0.21 Cr	Within contingency
Active Risks	1 (Monsoon delays)	Mitigated with buffer

9.2 Next Engine Triggers

Month 16: Structure topping out milestone. If on time: no engine trigger. If delayed > 15 days: milestone.delayed event triggers S-Curve rebase and Underwriter re-phase.

Month 22: FF&E procurement begins. Large committed costs will trigger Budget Variance recalculation.

Monthly: Macro indicator refresh (RBI repo rate, CPI, bond yields) triggers Factor Engine re-score and cascade to Decision Engine.

Month 36: Phase 2 gate assessment. All 8 criteria evaluated against actuals. phase2-gate.evaluated event triggers full engine cascade with updated room count and capex.

This document describes the architecture of a living investment operating system. The next step is to implement the service layer, event bus, and data persistence — the HTML prototype serves as a functional reference for the computation logic, which will be extracted into standalone services.